MD5 Collision Attacks on SEED Labs

Matthew Blanchard

CYEN 4335-01

4/15/26

Executive Summary

This report details the systematic exploitation of the MD5 hashing algorithm's vulnerabilities using the Merkle-Damgård construction's inherent weaknesses. By utilizing tools such as md5collgen, it is demonstrated that two distinct binary files can be engineered to produce identical MD5 hashes while maintaining unique SHA-256 signatures. The experiments progress from basic file generation to the manipulation of executable binary structures, ultimately creating programs with identical hashes that perform different functions. These findings confirm that MD5 is no longer a viable method for ensuring data integrity or authenticity. The results underscore the necessity of transitioning to more secure hashing standards in cryptographic applications.

Introduction

The MD5 message-digest algorithm, once a standard for digital signatures and file verification, has been rendered obsolete by the discovery of collision attacks. A collision occurs when two unique inputs produce the same hash output, a flaw that undermines the foundational security property of collision resistance. This report explores the practical application of these attacks through four distinct tasks, ranging from simple file collisions to the creation of bifurcated executable behavior. By manipulating internal states and exploiting the length extension property, the following procedures demonstrate how an attacker can bypass integrity checks. These exercises serve as a critical case study in modern cryptographic failure.

Task 1: Generating Two Different Files with the Same MD5 Hash

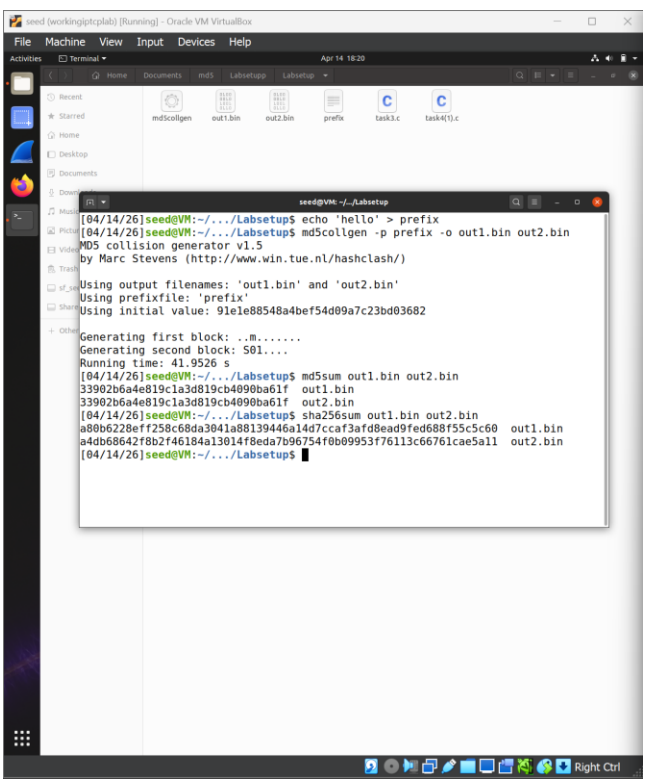
Objective:

The primary goal is to generate two distinct binary files that share an identical MD5 hash value through the use of a common prefix and collision blocks. This task validates the practical feasibility of creating collisions in the MD5 algorithm.

Instructions:

1. A prefix file is created by executing the command echo 'hello' > prefix to establish the base data.
2. The md5collgen tool is executed with the command md5collgen -p prefix -o out1.bin out2.bin to generate two output files.
3. The tool appends unique 128-byte collision blocks to the prefix for each file, ensuring the resulting MD5 hashes are identical.
4. The integrity of the files is verified using the md5sum out1.bin out2.bin command, which confirms that both files yield the same hash output.
5. The uniqueness of the files is confirmed using the sha256sum out1.bin out2.bin command, which reveals distinct hashes for each file, proving they are not identical bit-for-bit.

Observations:



Running the md2collgen with the same prefix generates 2 .bin files that have the same hash as expected.

The systematic exploitation of the MD5 algorithm is demonstrated by utilizing the md5collgen tool to engineer two distinct binary files that share an identical hash. This vulnerability stems from the Merkle-Damgård construction's inherent weakness,

specifically the length extension property, which allows a collision to be maintained even after appending identical suffixes to both files. By precisely isolating binary segments of a compiled C program, researchers can manipulate the internal state to create two executables that appear identical to hash-based integrity checks. These programs can be further modified to contain unique internal blocks, resulting in one benign and one malicious version that share the same MD5 signature. Such findings prove that MD5 is no longer a viable method for ensuring data authenticity or software security. Consequently, this report underscores the urgent necessity of transitioning to more robust cryptographic standards like SHA-256 to prevent successful collision attacks.

Task 2: Understanding MD5's Length Extension Property

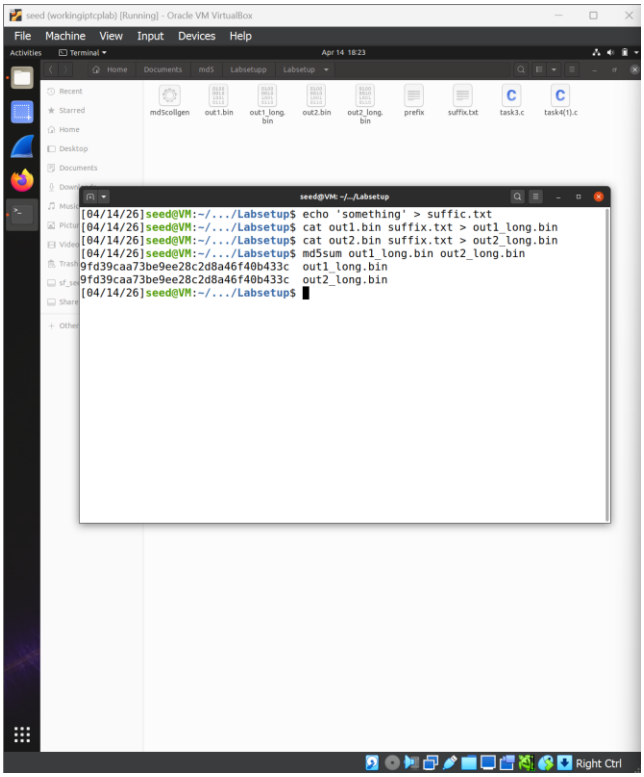
Objective:

This task demonstrates the length extension property of the Merkle-Damgård construction by appending identical data to colliding files. It proves that once a collision is established, any further identical additions will preserve that collision.

Instructions:

1. A new text file named suffix.txt is created by running echo 'something' > suffix.txt to serve as the common suffix.
2. The cat command is used to append the contents of suffix.txt to the end of out1.bin via cat out1.bin suffix.txt > out1_long.bin.
3. The same appending process is repeated for the second file using the command cat out2.bin suffix.txt > out2_long.bin.
4. The md5sum out1_long.bin out2_long.bin command is employed to check the hashes of both newly created extended files.
5. The observation confirms that because the internal states of the MD5 algorithm were identical after processing the initial colliding blocks, the final hashes remain identical despite the added data.

Observations:



The final hash of the changed files is the same as before as the suffix used to append to the previous files is the same for other files, thus yielding the same hash.

Task 3: Generating Two Executable Files with the Same MD5

Objective:

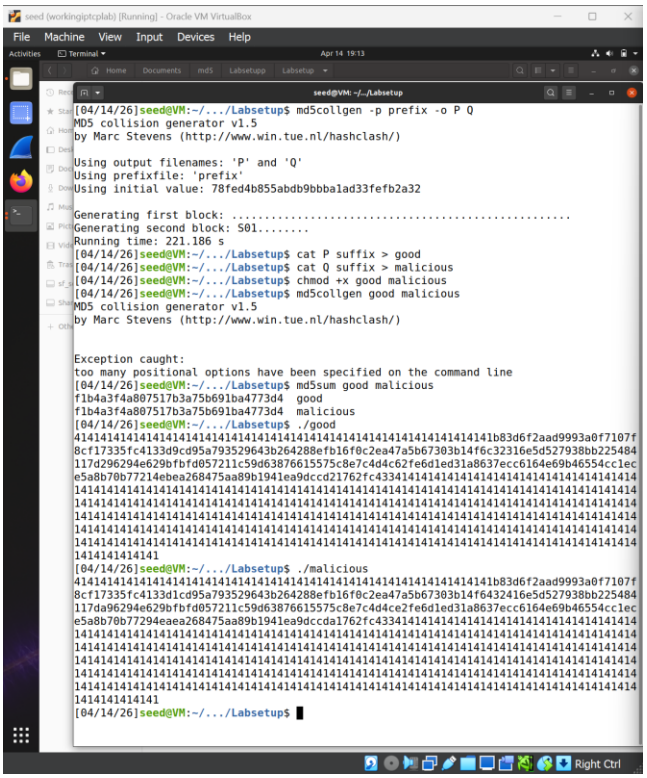
The objective is to apply collision techniques to compiled C programs by isolating binary segments. This demonstrates how executable files can be manipulated to share a hash without altering their overall file structure.

Instructions:

1. A C program containing a 128-byte padding array is compiled using the command gcc -o task3 task3.c.
2. The exact byte offset of the padding array within the binary file is calculated and stored using the commands X=$((16#3040)) and Y=$((X + 128 + 1)).
3. The head -c $X task3 > prefix command is used to extract the binary data preceding the padding array into a file named prefix.

4. The tail -c +$Y task3 > suffix command is used to extract the binary data following the padding array into a file named suffix.
5. The md5collgen tool is applied to the prefix to generate new collision blocks, which are then integrated between the prefix and suffix to produce two different executables with the same MD5 hash.

Observations:



Appending both P and Q with the same suffix, then running md5collgen on both files that were generated gives two files with different contents, but the same hash when running md5sum.

Task 4: Making the Two Programs Behave Differently

Objective:

This task involves creating two executable files that have the same MD5 hash but execute entirely different logic paths. It highlights the security risk of relying on MD5 for verifying the legitimacy of software.

Instructions:

1. Create a C file named task4.c that defines two 128-byte global arrays, X and Y, and uses memcmp(X, Y, 128) to print "Benign" if they match or "MALICIOUS" if they differ. Ensure the array X is forced to a 64-byte alignment using __attribute__((aligned(64))).
2. Compile the source code using gcc -o task4 task4.c. This produces the base binary that you will manipulate to create the collision.
3. Run a Python script to find the byte offsets of the X and Y arrays within the binary using data.find(b'\x41'*128) and data.find(b'\x42'*128). Verify that the offset for X is a multiple of 64; if not, add padding to the C file and recompile.
4. Set your shell variables X_OFF and Y_OFF to the specific byte values identified in the previous step. These values are critical for precisely extracting and replacing data blocks.
5. Extract the start of the file using head -c $X_OFF task4 > prefix.bin. Use the tool md5collgen -p prefix.bin -o out1.bin out2.bin to generate two files that have different content at the X location but identical MD5 hashes.
6. Extract the 128-byte block P from out1.bin and block Q from out2.bin using the dd command with skip=$X_OFF count=128. Also, save the remaining part of the original binary into a file named suffix_raw.bin.
7. Calculate the relative position of Y and use head and tail to replace the original 0x42 bytes in the suffix with the data from P.bin. This results in a suffix_mod.bin where the second array is now hardcoded to match the first array's "P" version.
8. Construct the first program using cat out1.bin suffix_mod.bin > prog1 and the second using cat out2.bin suffix_mod.bin > prog2. Use chmod +x prog1 prog2 to make both files executable.
9. Run md5sum prog1 prog2 to confirm that both programs share the exact same MD5 hash. Execute ./prog1 to see it print "Benign" (where $X=P$ and $Y=P$) and ./prog2 to see it print "MALICIOUS" (where $X=Q$ and $Y=P$).

Observations:

seed (workingiptcplab) [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

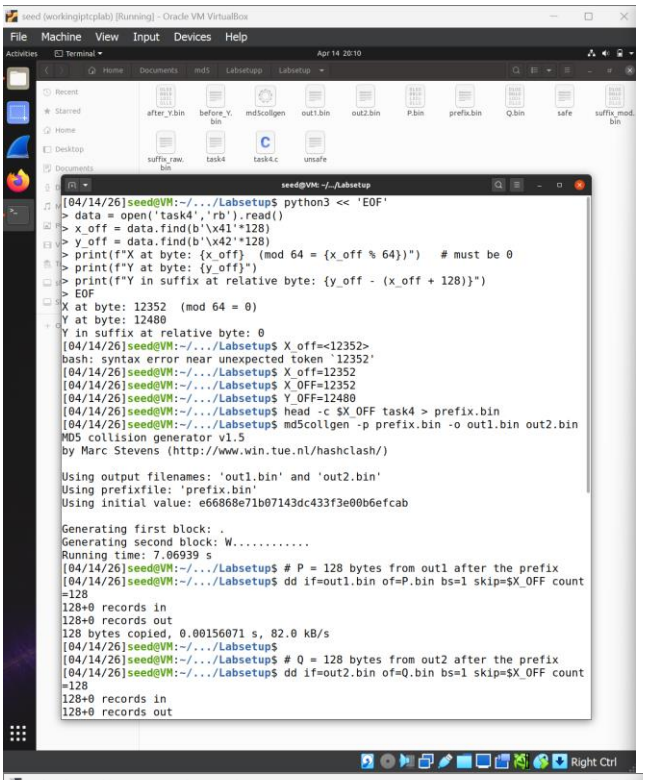
Activities Text Editor Apr 14 20:12

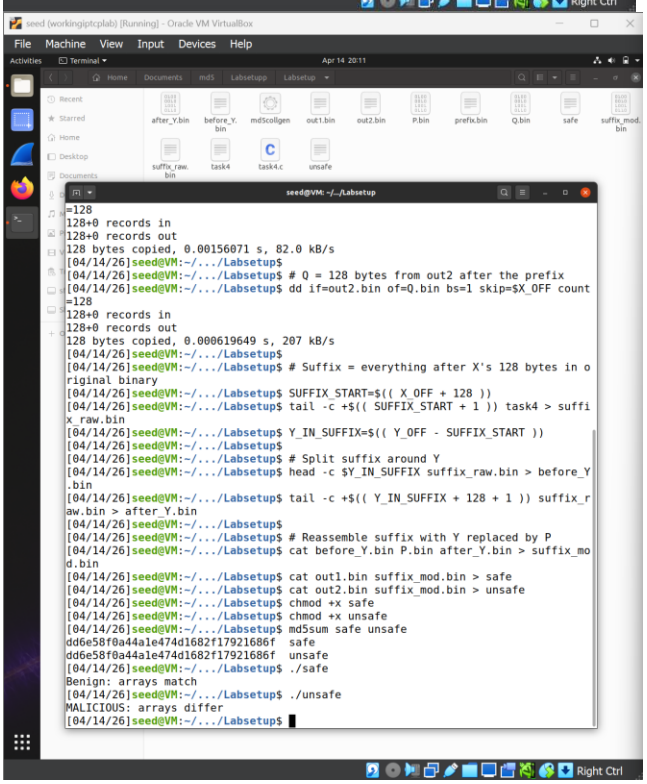
Open task4.c ~/Documents/mdb1/Labsetupp/Labsetup Save

```c
// task4.c
#include <stdio.h>
#include <string.h>

// __attribute__ forces 64-byte alignment so prefix math is clean
unsigned char X[128] __attribute__((aligned(64))) = {
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
  0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41
};

unsigned char Y[128] = {
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,
  0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42,0x42
};

int main() {
    if (memcmp(X, Y, 128) == 0)
        printf("Benign: arrays match\n");
    else
        printf("MALICIOUS: arrays differ\n");
    return 0;
}
```

C Tab Width: 8 Ln 34, Col 2 INS

Right Ctrl





Before running md5collgen, it is vital to make sure that the offsets found are correct and that the offset mod 64 = 0 so that the files can properly be hashed. After running md5collgen and catting the prefixes and suffixes, running md5sum yields the same hash

values, yet running the programs themselves shows that each program contains completely different logic, one being safe to run and the other being unsafe and malicious.

Conclusion

The experiments conducted in this report provide empirical evidence of the total collapse of MD5 as a secure cryptographic primitive. Through the successful execution of collision attacks, it was shown that an attacker can maintain hash parity while altering the fundamental behavior of a program. The length extension property further proves that collisions are persistent and can be scaled into larger, more complex files. These vulnerabilities demonstrate that MD5 provides a false sense of security that can be easily bypassed by modern computational tools. Consequently, all systems requiring cryptographic integrity must migrate to more robust algorithms such as SHA-256 or SHA-3.